

12 Topics You **MUST** Know Before Your Next AI / LLM Engineering Interview

RAG · Agents · Evals · Observability · Guardrails · Fine-tuning
Caching · Memory · Feedback Loops · Cost · Tool Calling · Prod Architecture

MUI. RAG (Retrieval-Augmented Generation)

🌀 **2.** Agents & Agentic Architecture

🌀 **3.** LLM / Agent Evaluation

🌀 **4.** Observability & Tracing

🌀 **5.** Guardrails & AI Security

🌀 **6.** Fine-tuning

🌀 **7.** Caching

🌀 **8.** Memory

🌀 **9.** Human / User Feedback Loops

🌀 **10.** LLM Cost & Performance Optimization

🌀 **11.** Tool Calling

🌀 **12.** Production Architecture

📖 concept + real example on every page — save this, you'll need it.

1. RAG (Retrieval-Augmented Generation)

Instead of relying only on what the LLM 'remembers' from training, we fetch fresh, relevant info at query time and stuff it into the prompt. Fixes hallucination + stale knowledge.

- ◆ **Chunking** — split docs into smaller pieces (fixed-size / recursive / semantic) — too big = noisy, too small = loses context
- ◆ **Embeddings** — convert text -> vectors that capture meaning (OpenAI, Cohere, BGE, ES)
- ◆ **Vector DBs** — store + search embeddings fast — Pinecone, Weaviate, Milvus, pgvector (ANN search: HNSW, IVF)
- ◆ **Hybrid Search** — dense (semantic) + sparse (BM25/keyword) combined -> better recall than either alone
- ◆ **Reranking** — cross-encoder re-scores top-k retrieved chunks for precision (e.g. Cohere Rerank)
- ◆ **Retrieval Quality** — measured via Precision@k, Recall@k, MRR
- ◆ **Context Compression** — summarize/filter retrieved chunks before sending to LLM -> saves tokens + cost

★ **EXAMPLE** (interview / DM ready)

Support bot for a food-delivery app's help docs: chunk the docs -> embed with BGE -> store in pgvector -> hybrid search (BM25 + vector) -> rerank top 5 -> compress to top 3 -> LLM answers WITH citation to the source doc.

★ **Tip:** Interviewer loves: 'RAG isn't just embeddings + vector DB — it's a whole retrieval PIPELINE with quality checks at each stage.'

2. Agents & Agentic Architecture

An agent = LLM that can reason, decide, act (call tools), observe the result, and loop — instead of just answering in one shot.

- ◆ **ReAct** — Thought -> Action -> Observation loop, repeated until task is done
- ◆ **Tool Calling** — LLM decides WHICH tool + WHAT args, based on the task
- ◆ **Planning** — break a big goal into sub-tasks — chain-of-thought / tree-of-thought / plan-and-execute
- ◆ **Memory** — short-term = context window, long-term = external store it can query
- ◆ **State** — tracking progress across multi-step tasks so the agent doesn't repeat/forget steps
- ◆ **Multi-Agent Systems** — specialised agents collaborate — e.g. planner + executor + critic
- ◆ **Workflows vs Agents** — workflow = FIXED deterministic steps; agent = LLM DYNAMICALLY decides next step

★ EXAMPLE (interview / DM ready)

A trip-planner agent: planner-agent splits query into flights + hotels + budget -> executor-agent calls flight/hotel APIs via ReAct -> critic-agent checks total cost stays under budget before returning the final itinerary.

★ **Tip:** Be ready to explain WHY you'd pick a workflow over an agent — workflows are more predictable + cheaper + easier to debug for well-defined tasks.

3. LLM / Agent Evaluation

How do you PROVE your AI feature actually works, before and after shipping?

- ◆ **Offline Evals** — run against a fixed 'golden' dataset BEFORE deployment (regression testing)
- ◆ **Online Evals** — monitor live traffic — A/B tests, real user feedback, production metrics
- ◆ **LLM-as-a-Judge** — use a strong LLM to score outputs (correctness, relevance)
— cheaper + faster than humans
- ◆ **Task Success** — binary/graded — did the agent actually complete the user's goal?
- ◆ **Faithfulness** — does the answer stay grounded in the retrieved context, or did it hallucinate?
- ◆ **Tool-Call Accuracy** — did the agent pick the RIGHT tool with the RIGHT parameters?

✧ **EXAMPLE** (interview / DM ready)

Evaluating a payments support agent: build 200 golden Q&A pairs -> GPT-4 as judge scores faithfulness 1-5 on each answer -> separately track tool-call accuracy weekly for the 'check_payment_status' tool.

★ **Tip:** Always mention BOTH offline + online evals — most candidates only talk about one and miss the full loop.

4. Observability & Tracing

If you can't SEE what your agent did step-by-step, you can't debug it or trust it in production.

- ◆ **Agent Traces** — full step-by-step log of a run — every thought, tool call, output
- ◆ **Tool Calls** — track input/output/latency/errors for EVERY tool invocation
- ◆ **Latency** — measure p50 / p95 / p99 across each stage of the pipeline, not just total
- ◆ **Token Usage** — input + output tokens per call -> direct cost tracking
- ◆ **Failure Tracking** — bucket failures — timeout, wrong tool picked, hallucination, schema error
- ◆ **Distributed Tracing** — correlate spans across microservices (OpenTelemetry) when the pipeline spans many services

★
EXAMPLE (interview / DM ready)

Using a tool like Langfuse/Arize on a delivery chatbot: trace every session end-to-end -> notice the reranker step adds 800ms at p95 -> fix by caching embeddings for repeat queries.

★ **Tip:** Mention a real tool by name (Langfuse, Arize, LangSmith, Datadog LLM Obs) — shows you've actually used this, not just read about it.

5. Guardrails & AI Security

LLMs follow instructions blindly — which is exactly the attack surface. Guardrails stop that from becoming a real-world incident.

- ◆ **Prompt Injection** — malicious instructions hijack the LLM — direct (user types it) or indirect (hidden inside a retrieved doc/webpage)
- ◆ **Jailbreaks** — bypassing safety rules via roleplay, encoding, or clever framing
- ◆ **Data Leakage** — LLM exposes system prompt, other users' data, or PII in its output
- ◆ **Tool Permissions** — least-privilege — an agent should NOT have write/delete access unless the task truly needs it
- ◆ **Output Validation** — schema/regex/format checks on agent output BEFORE it's executed or shown
- ◆ **PII Handling** — detect + redact personal data in both inputs and outputs (e.g. Presidio, regex rules)

★ **EXAMPLE** (interview / DM ready)

A payments-app support agent: sandbox all tool execution, strip PII before logging, and require an explicit user-confirmation step before any 'transfer_money' tool call — so a prompt-injection attack can't trigger a silent transfer.

★ **Tip:** The strongest answer here is always 'defense in depth' — no single guardrail, multiple layers (input filtering + sandboxing + output validation + human-in-the-loop for risky actions).

6. Fine-tuning

Changing the model's WEIGHTS (not just the prompt) so it behaves a certain way by default.

- ◆ **SFT** — Supervised Fine-Tuning — train on labelled input -> output pairs to shape style/format/domain behaviour
- ◆ **LoRA** — freeze the base model, train small low-rank 'adapter' matrices instead — MUCH cheaper than full fine-tuning
- ◆ **QLoRA** — LoRA + 4-bit quantization of the base model -> fine-tune large models even on a single consumer GPU
- ◆ **Prompting vs RAG vs Fine-tuning** — prompting = quick win, no data needed. RAG = knowledge changes often / lives outside the model. Fine-tune = you need consistent STYLE, FORMAT or behaviour that prompting/RAG can't reliably hold

★
EXAMPLE (interview / DM ready)

Fine-tuning a small open model with LoRA on 5,000 real Hinglish support chats so it ALWAYS replies in brand tone — instead of stuffing a 2-page system prompt into every request.

★ **Tip:** Classic interview trap: 'Should we fine-tune or use RAG?' — correct answer is usually 'depends on whether the problem is KNOWLEDGE (use RAG) or BEHAVIOUR/STYLE (fine-tune)', often both together.

7. Caching

LLM calls are slow + expensive — caching is the single highest-leverage optimization most teams under-use.

- ◆ **Semantic Caching** — cache by MEANING (embedding similarity) of the query, not exact text match — catches paraphrased questions
- ◆ **Prompt Caching** — provider-level caching of a repeated prompt PREFIX (system prompt + long context) to cut cost/latency
- ◆ **Response Caching** — store the final LLM output keyed by exact query hash
- ◆ **Embedding Caching** — don't re-embed documents that haven't changed
- ◆ **Cache Invalidation** — TTL-based, versioned, or event-based (invalidate the moment source data changes)

EXAMPLE (interview / DM ready)

A railway-booking chatbot permanently caches embeddings of static FAQ docs, uses semantic caching for common queries like 'how to cancel my ticket' (catches every paraphrase), and invalidates the moment the cancellation policy is updated.

★ **Tip:** Mention Anthropic/OpenAI's native prompt caching by name — most candidates only know 'Redis cache' and miss this layer entirely.

8. Memory

How an agent remembers things — both within one conversation and across many sessions.

- ◆ **Short-term / Context Memory** — whatever fits inside the current context window for THIS session
- ◆ **Long-term Memory** — persisted across sessions in a DB/vector store, pulled back in when relevant
- ◆ **Episodic Memory** — specific past EVENTS — 'user asked about X on this date'
- ◆ **Semantic Memory** — general FACTS/preferences learned about the user or domain over time
- ◆ **Memory Retrieval** — fetching the right memories via similarity search, or recency + importance scoring

★
EXAMPLE (interview / DM ready)

A finance assistant remembers (semantic memory) that a user prefers monthly budget summaries, and recalls (episodic memory) a specific past complaint about a merchant when that merchant comes up again.

★ **Tip:** Draw the line clearly between 'context window' and 'memory system' — a lot of candidates confuse the two.

9. Human / User Feedback Loops

Real users are the best (and cheapest) source of eval data — IF you have a system to actually capture and use it.

- ◆ **Explicit Feedback** — thumbs up/down, star ratings, direct corrections
- ◆ **Implicit Feedback** — behavioural signals — click-through, session abandonment, how much a user edits a suggestion
- ◆ **Reward Signals** — feedback converted into a scalar reward, used for RLHF-style fine-tuning
- ◆ **Evaluation Datasets** — worst-performing real interactions get curated into golden eval sets
- ◆ **Continuous Improvement** — flagged failures feed back into eval sets AND fine-tuning data — a closed loop

EXAMPLE (interview / DM ready)

An order-support bot logs every thumbs-down; each week the worst 50 get turned into new golden eval cases so the same mistake can never silently regress back in.

★ **Tip:** The keyword interviewers want to hear is 'closed loop' — feedback that doesn't feed back into evals/training is just a vanity metric.

10. LLM Cost & Performance Optimization

At scale, a few cents per call becomes a real budget line — this is where senior engineers stand out.

- ◆ **Token Reduction** — shorter prompts, drop redundant context, compress history
- ◆ **Model Routing** — send easy queries to a small/cheap model, hard ones to a large model (cascade/router pattern)
- ◆ **Batching** — group multiple requests together for throughput, especially for embeddings/offline jobs
- ◆ **Caching** — reuse prior computation wherever possible (see Topic 7)
- ◆ **Context Compression** — summarize long history/retrieved docs before sending to the LLM
- ◆ **Smaller Models** — distillation or a fine-tuned small model for one narrow task instead of a big general model

EXAMPLE (interview / DM ready)

A system-design Q&A bot routes simple FAQ-type queries to a small fine-tuned model, and only escalates genuinely complex architecture questions to GPT-4 — cutting overall cost by roughly 60%.

★ **Tip:** Model routing is the single most 'senior-sounding' answer here — mention it even if you also mention caching.

11. Tool Calling

The mechanism that lets an LLM actually DO things in the real world, not just talk.

◆ **Function Calling** — LLM outputs structured JSON matching a function's schema instead of free text

◆ **Schemas** — strict definition of a tool's name + params + types — this is the 'contract'

◆ **Validation** — verify the LLM's generated arguments against the schema BEFORE executing — catches hallucinated params

◆ **Retries** — retry a failed tool call with backoff, optionally re-prompting the LLM with the error message

◆ **Timeouts** — cap how long a tool is allowed to run, so the agent loop never hangs forever

◆ **Permissions** — scope exactly which tools/agents can access what — read vs write access

◆ **Idempotency** — repeated calls (from retries) must NOT cause duplicate side-effects — use idempotency keys

✈
EXAMPLE (interview / DM ready)

A 'book_cab' tool uses an idempotency key per booking request, so a network-retry after a timeout can't accidentally book two cabs for the same trip.

★ **Tip:** Idempotency is the detail most candidates forget — bringing it up unprompted is a strong signal you've built real production tool-calling systems.

12. Production Architecture

How all of this actually holds up under real traffic, not just in a notebook demo.

- ◆ **Queues** — decouple request intake from processing — Kafka/SQS — so spikes don't take down the system
- ◆ **Async Execution** — long-running agent tasks run in the background; client polls or gets a webhook when done
- ◆ **State Management** — persist agent/task state (Redis/DB) across steps and retries
- ◆ **Retries & Fallbacks** — retry transient failures; fall back to a simpler model or cached response if the main one degrades
- ◆ **Rate Limits** — token-bucket / leaky-bucket to protect both LLM API quota AND downstream systems
- ◆ **Scalability** — horizontally scale worker pools, autoscale based on queue depth

★ **EXAMPLE** (interview / DM ready)

An AI team-suggestion feature queues requests via SQS during match-day traffic spikes; workers process them asynchronously, and if the LLM service degrades, the system falls back to a simple rule-based suggester instead of failing outright.

★ **Tip:** Always pair 'retries' with 'fallbacks' in your answer — retries alone don't help if the whole LLM provider is down.